IoT 1

Les 2 – input, output, AD-converter, …

Vooraf:
wat is een hexadecimaal getal?

Wat is een hexadecimaal getal?

- **Decimaal**
  - **Dagelijkse** wereld = 10-delig stelsel
  - 10 vingers
  - Decimaal stelsel
  - Zie eerste leerjaar ;-)
  - Grondtal = **10**
  - $5 . 10^2 + 0 . 10^1 + 8 . 10^0 = 508$ of $(508)_{10}$

Wat is een hexadecimaal getal?

- **Binair**
  - **computerwereld** = 2-delig stelsel
  - Enkel 0 en 1 toegelaten
  - Binair stelsel
  - Grondtal = **2**
  - $(1001\ 1110)_2$

    $= 1.2^7+0.2^6+0.2^5+1.2^4+1.2^3+1.2^2+1.2^1+0.2^0$

    $= 158$ of $(158)_{10}$

Wat is een hexadecimaal getal?

- **Hexadecimaal**
  - **computerwereld** = 16-delig stelsel
  - Enkel 0 tot en met F toegelaten
  - Hexadecimaal stelsel
  - Grondtal = **16**
  - $(89)_{hex} = 8.16^1+9.16^0 = (137)_{10} = (1000\ 1001)_2$

hogeschool vives

Wat is een hexadecimaal getal?

Decimaal	Binair	Hexadecimaal
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7

Decimaal	Binair	Hexadecimaal
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

Bijvoorbeeld:

$(137)_{10} = (1000\ 1001)_2 = (89)_{hex}$ of 0x89
$(619247740)_{10} = (0010\ 0100\ 1110\ 1000\ 1111\ 1000\ 0111\ 1100)_2 = (24E8F87C)_{hex}$ of 0x24E8F87C

Vooraf:
basis digitale poorten

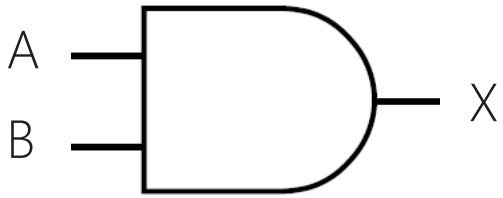
Basis digitale poorten

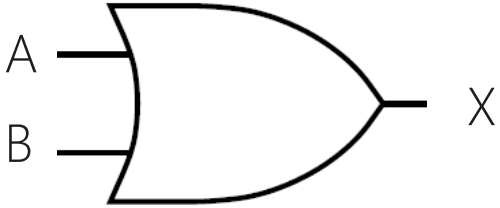
AND		
A	B	X
0	0	0
0	1	0
1	0	0
1	1	1

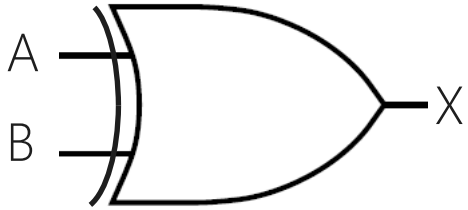
OR		
A	B	X
0	0	0
0	1	1
1	0	1
1	1	1

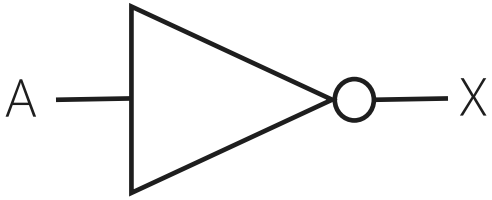
EXOR		
A	B	X
0	0	0
0	1	1
1	0	1
1	1	0

NOT	
A	X
0	1
1	0









Eenvoudige output genereren

Eenvoudige output genereren

- Een pin als **output**
  - Na **reset** => alle pinnen staan op ‘**input**’
  - Daardoor zijn ze ‘**hoog-ohmig**’ en kunnen ze weinig schade aanrichten.
  - Om een pin ‘output’ te maken, moeten we enkele **instellingen doen**.
  - De ‘instellingen’ staan **in zogenaamde ‘registers’.** Daarvan vind je de info in de **datasheets**…

Eenvoudige output genereren

- Een pin als **output**
  - Stel dat we **LED1** op output willen zetten…

Verwijzing naar het Advanced High-performance Bus clock ENable Register.

```
16  // Clock voor GPIOC inschakelen.
17  RCC->AHBENR = RCC->AHBENR | RCC_AHBENR_GPIOCEN;
18
19  // LED1 op output zetten.
20  GPIOC->MODER = (GPIOC->MODER & ~GPIO_MODER_MODER0) | GPIO_MODER_MODER0_0;
```

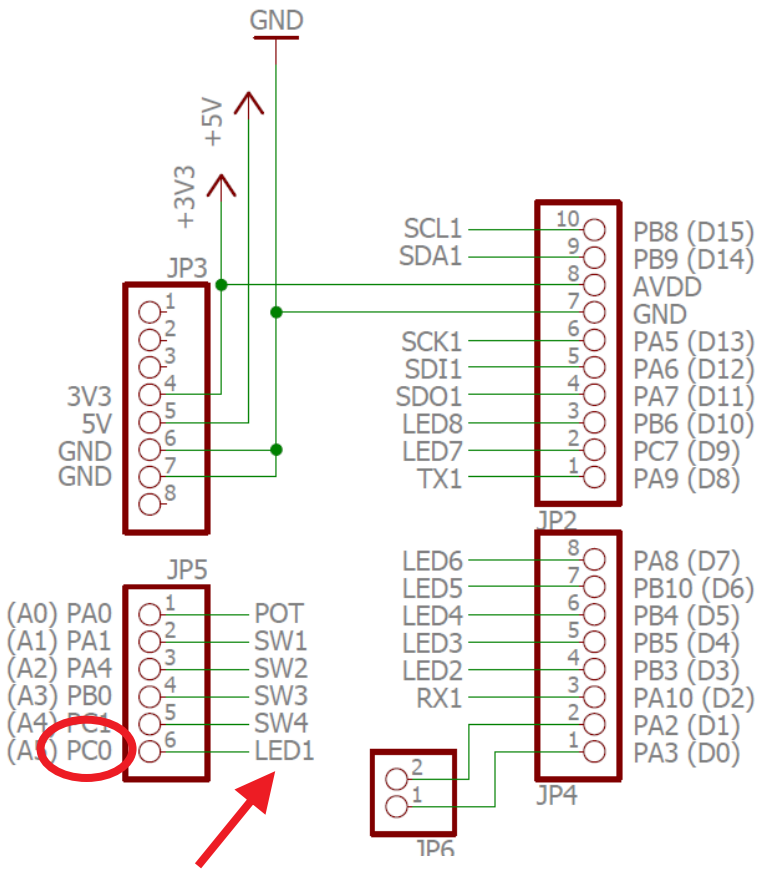
Verwijzing naar het MODE Register van poort C.

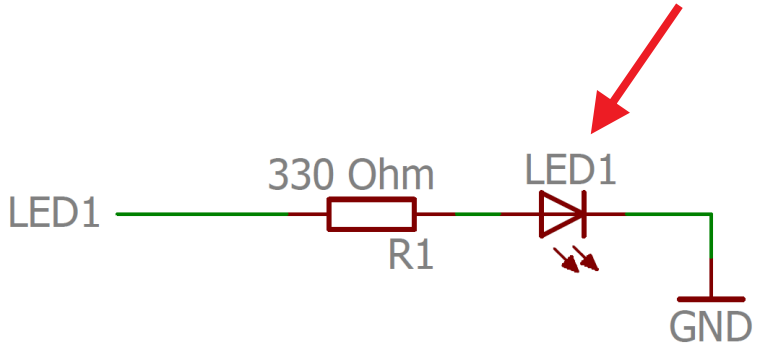
'Bitwise operations' via AND (&) OR (|) en NOT (~).

hogeschool vives

Eenvoudige output genereren

- Een pin als **output**
  - Neem het **schema** van het Nucleo Extension Shield erbij.





Een digitale '1' op PC0, zorgt voor 3,3V op het draadje met naam 'LED1'. Dat zorgt op zijn beurt voor een stroom(pje) door de LED, waardoor die **oplicht**....!

Eenvoudige output genereren

- Een pin als **output**
  - Neem de **reference manual** van de STM32F091RC erbij.

6.4.6 AHB peripheral clock enable register (RCC_AHBENR)

Address offset: 0x14

Reset value: 0x0000 0014

Access: no wait state, word, half-word and byte access

Note: When the peripheral clock is not active, the peripheral register values may not be readable by software and the returned value is always 0x0.

Hier kan je de klok inschakelen voor poort C...

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	TSCEN	Res.	IOPF EN	IOPE EN	IOPD EN	IOPC EN	IOPB EN	IOPA EN	Res.
							rw		rw	rw	rw	1	rw	rw	

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CRC EN	Res.	FLITF EN	Res.	SRAM EN	DMA2 EN	DMA EN
									rw		rw		rw	rw	rw

Bron: STM32F0 Reference Manual – RM0091 (st.com)

hogeschool vives

Eenvoudige output genereren

- Een pin als **output**
  - Neem de **reference manual** van de STM32F091RC erbij.

8.4.1 GPIO port mode register (GPIOx_MODER) (x =A..F)

Hier kan je de pin op output zetten...

Address offset:0x00

Reset values:

- 0x2800 0000 for port A
- 0x0000 0000 for other ports

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	0	1

Bits 2y+1:2y **MODERy[1:0]:** Port x configuration bits (y = 0..15)
These bits are written by software to configure the I/O mode.
00: Input mode (reset state)
01: General purpose output mode
10: Alternate function mode
11: Analog mode

Bron: STM32F0 Reference Manual – RM0091 (st.com)

Eenvoudige output genereren

- Een pin als **output**
  - **Waarom** gebeurt voorgaande?

AHBENR is een verwijzing naar een 32-bit getal (in het microcontrolllergeheugen).

```
16  // Clock voor GPIOC inschakelen.
17  RCC->AHBENR = RCC->AHBENR | RCC_AHBENR_GPIOCEN;
18
19  // LED1 op output zetten.
20  GPIOC->MODER = (GPIOC->MODER & ~GPIO_MODER_MODER0) | GPIO_MODER_MODER0_0;
```

OPM: het AHBENR-register wordt beschikbaar gesteld via een Struct (RCC), maar dat bekijken we later…

Eenvoudige output genereren

- Een pin als **output**
  - **Waarom** gebeurt voorgaande?

RCC_AHBENR_GPIOCEN is ook een **32-bit getal**. De waarde is: 0x00080000.

```
16  // Clock voor GPIOC inschakelen.
17  RCC->AHBENR = RCC->AHBENR | RCC_AHBENR_GPIOCEN;
18
19  // LED1 op output zetten.
20  GPIOC->MODER = (GPIOC->MODER & ~GPIO_MODER_MODER0) | GPIO_MODER_MODER0_0;
```

OPM: je kan de waardes van de getallen (defines) vinden via ‘Go to definition’ in µVision… Doen!

Eenvoudige output genereren

- Een pin als **output**
  - **Waarom** gebeurt voorgaande?

De bewerking is een '**bitwise OR**' voor twee 32-bit getallen.

```
16  // Clock voor GPIOC inschakelen.
17  RCC->AHBENR = RCC->AHBENR | RCC_AHBENR_GPIOCEN;
18
19  // LED1 op output zetten.
20  GPIOC->MODER = (GPIOC->MODER & ~GPIO_MODER_MODER0) | GPIO_MODER_MODER0_0;
```

Door de bitwise OR met het beschikbare getal, weet je zeker dat vakje 19 van het AHBENR-register op één komt te staan! **Prachtig**, dat is wat we nodig hebben om poort C in te schakelen.

Eenvoudige output genereren

- Een pin als **output**
  - **Waarom** gebeurt voorgaande?

MODER is een verwijzing naar een 32-bit getal (in het microcontrolllergeheugen).

```
16  // Clock voor GPIOC inschakelen.
17  RCC->AHBENR = RCC->AHBENR | RCC_AHBENR_GPIOCEN;
18
19  // LED1 op output zetten.
20  GPIOC->MODER = (GPIOC->MODER & ~GPIO_MODER_MODER0) | GPIO_MODER_MODER0_0;
```

OPM: het MODER-register wordt beschikbaar gesteld via een Struct (RCC), maar dat bekijken we later…

Eenvoudige output genereren

- Een pin als **output**
  - **Waarom** gebeurt voorgaande?

GPIO_MODER_MODER0 is ook een **32-bit getal**. De waarde is: 0x00000003.

```
16  // Clock voor GPIOC inschakelen.
17  RCC->AHBENR = RCC->AHBENR | RCC_AHBENR_GPIOCEN;
18
19  // LED1 op output zetten.
20  GPIOC->MODER = (GPIOC->MODER & ~GPIO_MODER_MODER0) | GPIO_MODER_MODER0_0;
```

OPM: je kan de waardes van de getallen (defines) vinden via ‘Go to definition’ in µVision… Doen!

Eenvoudige output genereren

- Een pin als **output**
  - **Waarom** gebeurt voorgaande?

De bewerking is een '**bitwise AND**' van het geïnverteerde, voor twee 32-bit getallen.

```
16  // Clock voor GPIOC inschakelen.
17  RCC->AHBENR = RCC->AHBENR | RCC_AHBENR_GPIOCEN;
18
19  // LED1 op output zetten.
20  GPIOC->MODER = (GPIOC->MODER & ~GPIO_MODER_MODER0) | GPIO_MODER_MODER0_0;
```

Door de bitwise AND **met het geïnverteerde** beschikbare getal, weet je zeker dat vakjes 0 en 1 van het MODER-register op nul komen te staan! Dat is wat we nodig hebben als eerste stap.

Eenvoudige output genereren

- Een pin als **output**
  - **Waarom** gebeurt voorgaande?

GPIO_MODER_MODER0_0 is ook een 32-bit getal. De waarde is: 0x00000001.

```
16  // Clock voor GPIOC inschakelen.
17  RCC->AHBENR = RCC->AHBENR | RCC_AHBENR_GPIOCEN;
18
19  // LED1 op output zetten.
20  GPIOC->MODER = (GPIOC->MODER & ~GPIO_MODER_MODER0) | GPIO_MODER_MODER0_0;
```

OPM: je kan de waardes van de getallen (defines) vinden via ‘Go to definition’ in µVision… Doen!

Eenvoudige output genereren

- Een pin als **output**
  - **Waarom** gebeurt voorgaande?

De bewerking is een '**bitwise OR**' voor twee 32-bit getallen.

```
16  // Clock voor GPIOC inschakelen.
17  RCC->AHBENR = RCC->AHBENR | RCC_AHBENR_GPIOCEN;
18
19  // LED1 op output zetten.
20  GPIOC->MODER = (GPIOC->MODER & ~GPIO_MODER_MODER0) | GPIO_MODER_MODER0_0;
```

Door de bitwise OR met het beschikbare getal, weet je zeker dat vakje 0 van het MODER-register op één komt te staan! Dat is wat we nodig hebben als laatste stap om poort C, pin 0 als uitgang te zetten.

Eenvoudige output genereren

- Een pin als **output**
  - Heeft ook een maximum stroom.

    25mA per pin en 80mA in totaal
  - Heeft ook een maximum **schakelsnelheid**.

    In te stellen via het OSPEEDR-register.

    Kan effect hebben op **EMC**

    (ElectroMagnetic Compatibility).
  - **Hoe** zou je LED1 in- en uitschakelen?

```
GPIOC->ODR =
GPIOC->ODR =
```


hogeschool vives

Eenvoudige output genereren

- Een pin als **output**
  - Hoe zou je een **knipper-LED** kunnen maken met zo weinig mogelijk instructies?

```
// Oneindig lang lopen...
while (1)
{
  // LED1 toggle'n
  GPIOC->ODR = GPIOC->ODR ^ GPIO_ODR_0;

  // een tijdje wachten
  for(ticks = 0; ticks < 2000000; ticks++)
    __NOP();
}
```

'Bitwise EXOR operation'.

__NOP() is een '**No Operation**'. Het zorgt ervoor dat de CPU één klokcyclus 'niets' doet…

Eenvoudige input verwerken

Eenvoudige input verwerken

- Een pin als **input**
  - Na **reset** => alle pinnen staan **reeds op ‘input’…**
  - Daardoor zijn ze ‘**hoog-ohmig**’ en kunnen ze weinig schade aanrichten.
  - Om een pin ‘input’ te maken, moeten we dus **niets doen behalve kloksignaal** voorzien voor de bewuste poort.
  - De ‘instellingen’ na reset, staan in zogenaamde ‘registers’. Daarvan vind je de info in de datasheets…
  - Als je na verloop van tijd een output pin toch opnieuw input wil maken, moet je **alsnog het MODER-register instellen**.

Eenvoudige input verwerken

- Een pin als **input**
  - Stel dat we **SW1** op input willen zetten…

Verwijzing naar het Advanced High-performance Bus clock ENable Register.

```
// Clock voor GPIOA inschakelen.
RCC->AHBENR = RCC->AHBENR | RCC_AHBENR_GPIOAEN;

// Alle pinnen met drukknoppen verbonden, op input zetten.
GPIOA->MODER = GPIOA->MODER & ~GPIO_MODER_MODER1;
```

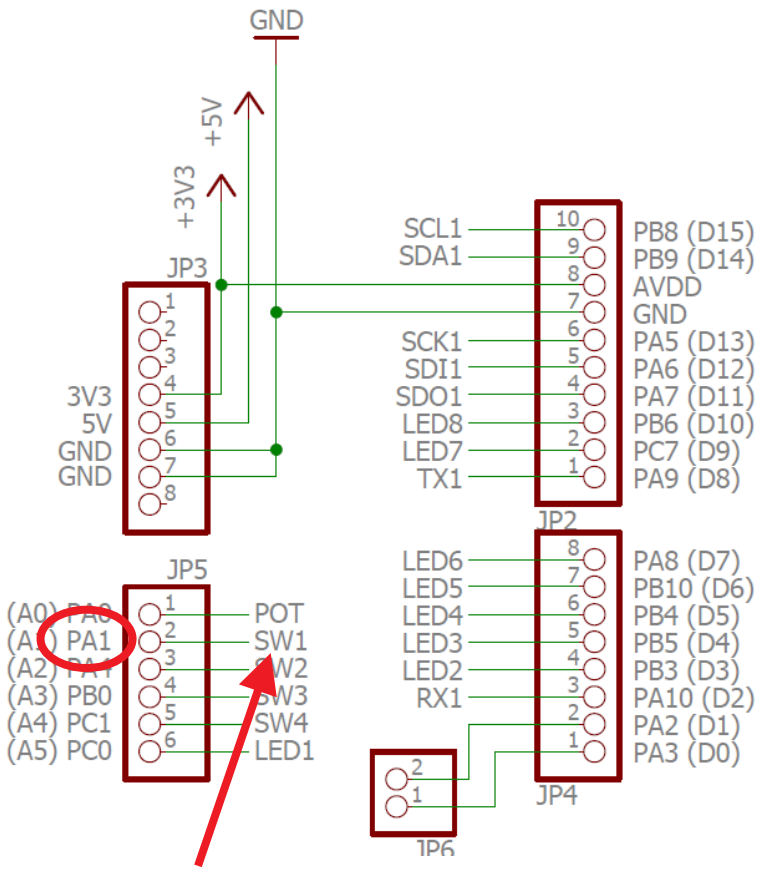
Verwijzing naar het MODE Register van poort A.

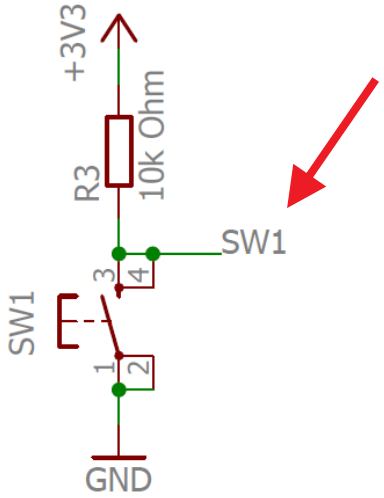
'Bitwise operations' via AND (&) en NOT (~).

hogeschool vives

Eenvoudige input verwerken

- Een pin als **input**
  - Neem het **schema** van het Nucleo Extension Shield erbij.





Drukken op de knop, zorgt voor 0V op het draadje 'SW1'. Niet drukken op de knop, zorgt voor een 3,3V op datzelfde draadje (via de pull-up resistor). Daarom spreken we van een '**actief lage drukknop**'!

Eenvoudige input verwerken

- Een pin als **input**
  - Neem de **reference manual** van de STM32F091RC erbij.

6.4.6 AHB peripheral clock enable register (RCC_AHBENR)

Hier kan je de klok inschakelen voor poort A...

Address offset: 0x14

Reset value: 0x0000 0014

Access: no wait state, word, half-word and byte access

Note: When the peripheral clock is not active, the peripheral register values may not be readable by software and the returned value is always 0x0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	TSCEN	Res.	IOPF EN	IOPE EN	IOPD EN	IOPC EN	IOPB EN	IOPA EN	Res.
							rw		rw	rw	rw	rw	rw	1	

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CRC EN	Res.	FLITF EN	Res.	SRAM EN	DMA2 EN	DMA EN
									rw		rw		rw	rw	rw

Bron: STM32F0 Reference Manual – RM0091 (st.com)

hogeschool vives

Eenvoudige input verwerken

- Een pin als **input**
  - Neem de **reference manual** van de STM32F091RC erbij.

8.4.1 GPIO port mode register (GPIOx_MODER) (x =A..F)

Hier kan je de pin op input zetten...

Address offset:0x00

Reset values:

- 0x2800 0000 for port A
- 0x0000 0000 for other ports

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	0	0	rw	rw

Bits 2y+1:2y **MODERy[1:0]:** Port x configuration bits (y = 0..15)
These bits are written by software to configure the I/O mode.
00: Input mode (reset state)
01: General purpose output mode
10: Alternate function mode
11: Analog mode

Bron: STM32F0 Reference Manual – RM0091 (st.com)

Eenvoudige input verwerken

- Een pin als **input**
  - **Waarom** gebeurt voorgaande?

AHBENR is een verwijzing naar een 32-bit getal (in het microcontrolllergeheugen).

```
// Clock voor GPIOA inschakelen.
RCC->AHBENR = RCC->AHBENR | RCC_AHBENR_GPIOAEN;

// Alle pinnen met drukknoppen verbonden, op input zetten.
GPIOA->MODER = GPIOA->MODER & ~GPIO_MODER_MODER1;
```

OPM: het AHBENR-register wordt beschikbaar gesteld via een Struct (RCC), maar dat bekijken we later…

Eenvoudige input verwerken

- Een pin als **input**
  - **Waarom** gebeurt voorgaande?

RCC_AHBENR_GPIOAEN is ook een 32-bit getal. De waarde is: 0x00020000.

```
// Clock voor GPIOA inschakelen.
RCC->AHBENR = RCC->AHBENR | RCC_AHBENR_GPIOAEN;

// Alle pinnen met drukknoppen verbonden, op input zetten.
GPIOA->MODER = GPIOA->MODER & ~GPIO_MODER_MODER1;
```

OPM: je kan de waardes van de getallen (defines) vinden via 'Go to definition' in µVision… Doen!

Eenvoudige input verwerken

- Een pin als **input**
  - **Waarom** gebeurt voorgaande?

De bewerking is een ‘bitwise OR’ voor twee 32-bit getallen.

```
11  // Clock voor GPIOA inschakelen.
12  RCC->AHBENR = RCC->AHBENR | RCC_AHBENR_GPIOAEN;
13
14  // Alle pinnen met drukknoppen verbonden, op input zetten.
15  GPIOA->MODER = GPIOA->MODER & ~GPIO_MODER_MODER1;
```

Door de bitwise OR met het beschikbare getal, weet je zeker dat vakje 17 van het AHBENR-register op één komt te staan! Prachtig, dat is wat we nodig hebben om poort A in te schakelen.

Eenvoudige input verwerken

- Een pin als **input**
  - **Waarom** gebeurt voorgaande?

MODER is een verwijzing naar een 32-bit getal (in het microcontrolllergeheugen).

```
// Clock voor GPIOA inschakelen.
RCC->AHBENR = RCC->AHBENR | RCC_AHBENR_GPIOAEN;

// Alle pinnen met drukknoppen verbonden, op input zetten.
GPIOA->MODER = GPIOA->MODER & ~GPIO_MODER_MODER1;
```

OPM: het MODER-register wordt beschikbaar gesteld via een Struct (RCC), maar dat bekijken we later…

Eenvoudige input verwerken

- Een pin als **input**
  - **Waarom** gebeurt voorgaande?

GPIO_MODER_MODER1 is ook een 32-bit getal. De waarde is: 0x0000000C.

```
11  // Clock voor GPIOA inschakelen.
12  RCC->AHBENR = RCC->AHBENR | RCC_AHBENR_GPIOAEN;
13
14  // Alle pinnen met drukknoppen verbonden, op input zetten.
15  GPIOA->MODER = GPIOA->MODER & ~GPIO_MODER_MODER1;
```

OPM: je kan de waardes van de getallen (defines) vinden via 'Go to definition' in µVision… Doen!

Eenvoudige input verwerken

- Een pin als **input**
  - **Waarom** gebeurt voorgaande?

De bewerking is een ‘bitwise AND’ van het geïnverteerde, voor twee 32-bit getallen.

```
11  // Clock voor GPIOA inschakelen.
12  RCC->AHBENR = RCC->AHBENR | RCC_AHBENR_GPIOAEN;
13
14  // Alle pinnen met drukknoppen verbonden, op input zetten.
15  GPIOA->MODER = GPIOA->MODER & ~GPIO_MODER_MODER1;
```

Door de bitwise AND met het geïnverteerde beschikbare getal, weet je zeker dat vakjes 2 en 3 van het MODER-register op nul komen te staan! Dat is wat we nodig hebben om een input te maken.

Eenvoudige input verwerken

- Een pin als **input**
  - **Hoe** zou je de toestand van een knop kunnen **detecteren**?

De if-voorwaarde is een ‘bitwise AND’ van twee 32-bit getallen, die vergeleken wordt met opnieuw een 32-bit getal (met waarde 0x00000002).

```
27 bool SW1Active(void)
28 {
29   // SW1 actief?
30   if((GPIOA->IDR & GPIO_IDR_1) != GPIO_IDR_1)
31       return true;
32   else
33       return false;
34 }
```

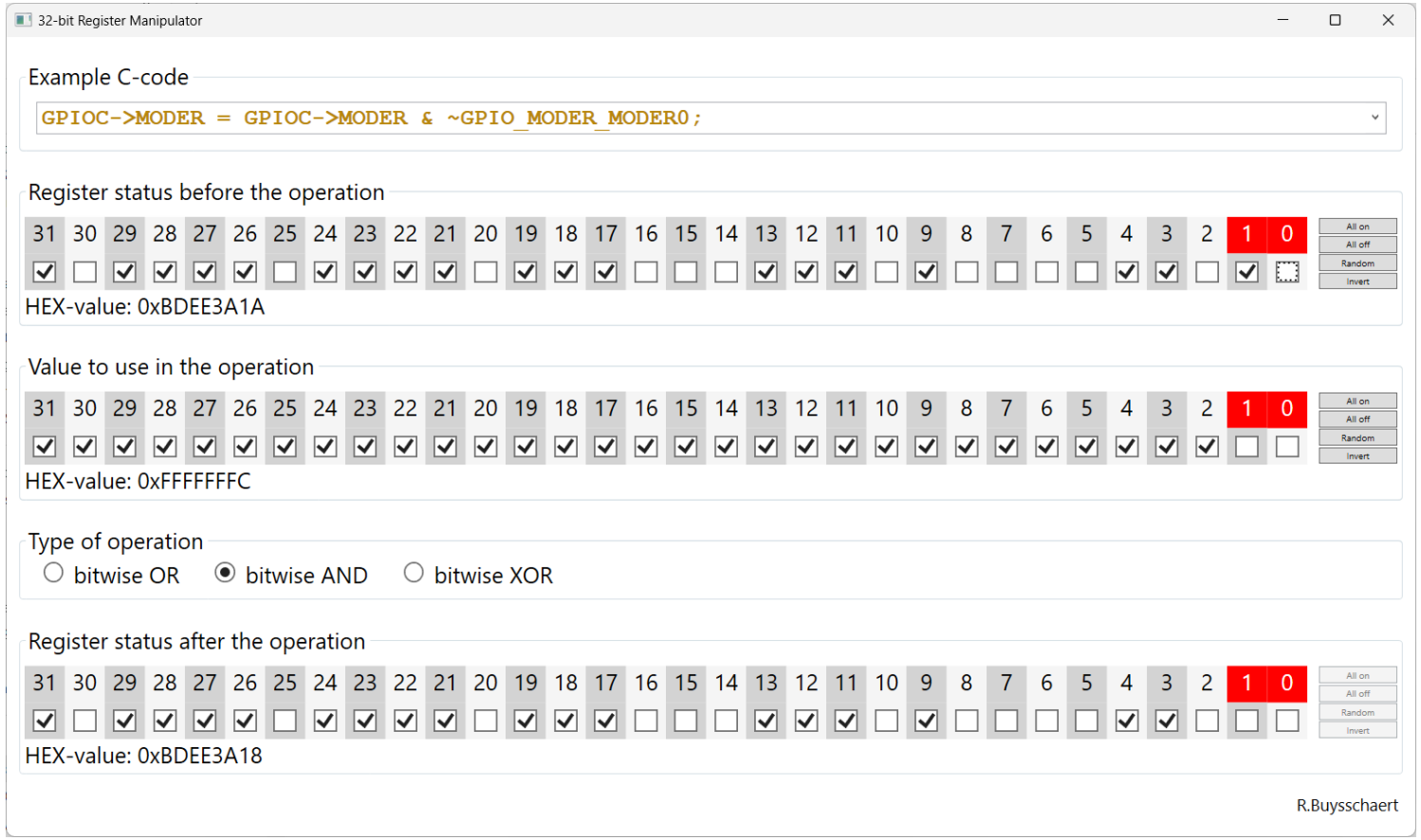
Merk op dat je een soort ‘**masker**’ legt over GPIOA->IDR, door middel van de bitwise AND…

Zijn het linker en rechter deel niet gelijk aan elkaar, keer dan terug met ‘true’.

ARM Register Manipulator

ARM Register Manipulator

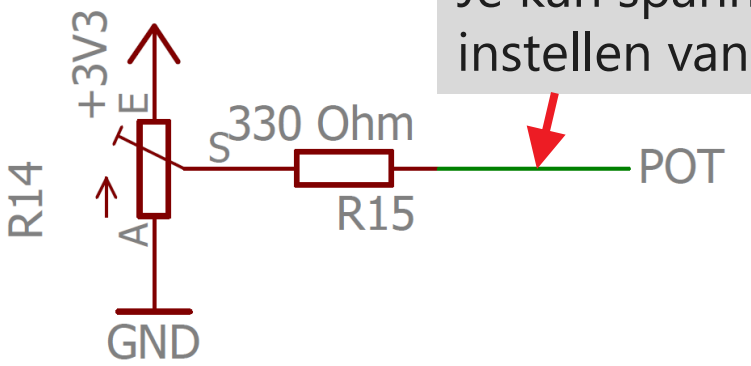
- Om te **oefenen** op de ‘bitwise operations’, kan je eventueel gebruik maken van een **simulatie**programmaatje. Download het via Toledo…



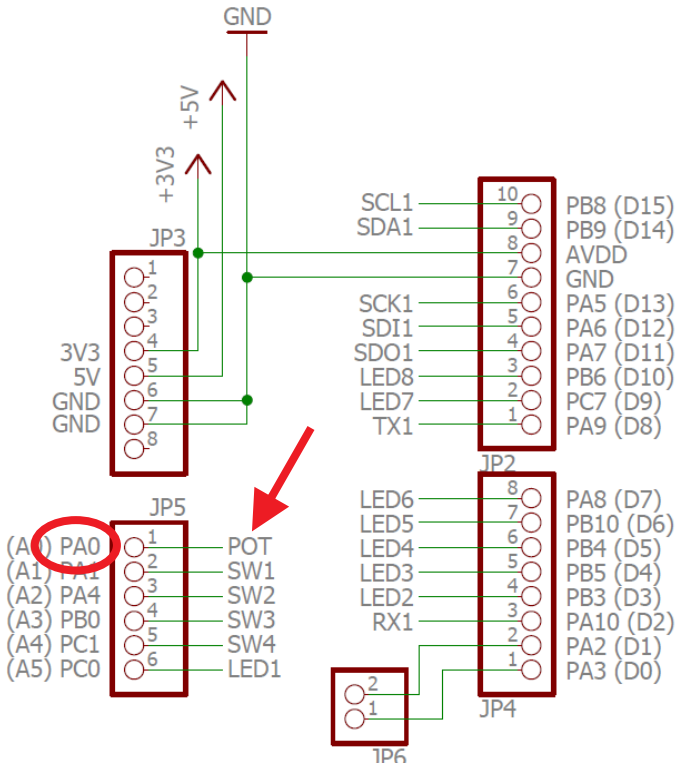
AD-converter

AD-converter

- **Wat** is een AD-converter?
  - Een Analoog-naar-Digitaal-omzetter.
  - Analoge **spanning vertalen naar** een evenwaardig digitaal **getal**.
  - Testen via trimmer op het NES.



Je kan spanningen instellen van 0V tot 3,3V.



R15 zorgt voor **stroombeveiliging**, wanneer PA0 verkeerdelijk op output zou staan…

AD-converter

- **Wat** is een AD-converter?
  - De AD-converter(s) in onze STM32F091RC, hebben een **12-bit** nauwkeurigheid…
  - De spanningen tussen 0 en 3V3 worden dus opgesplitst in 4096 gelijke stukjes (met **4095** als maximum).

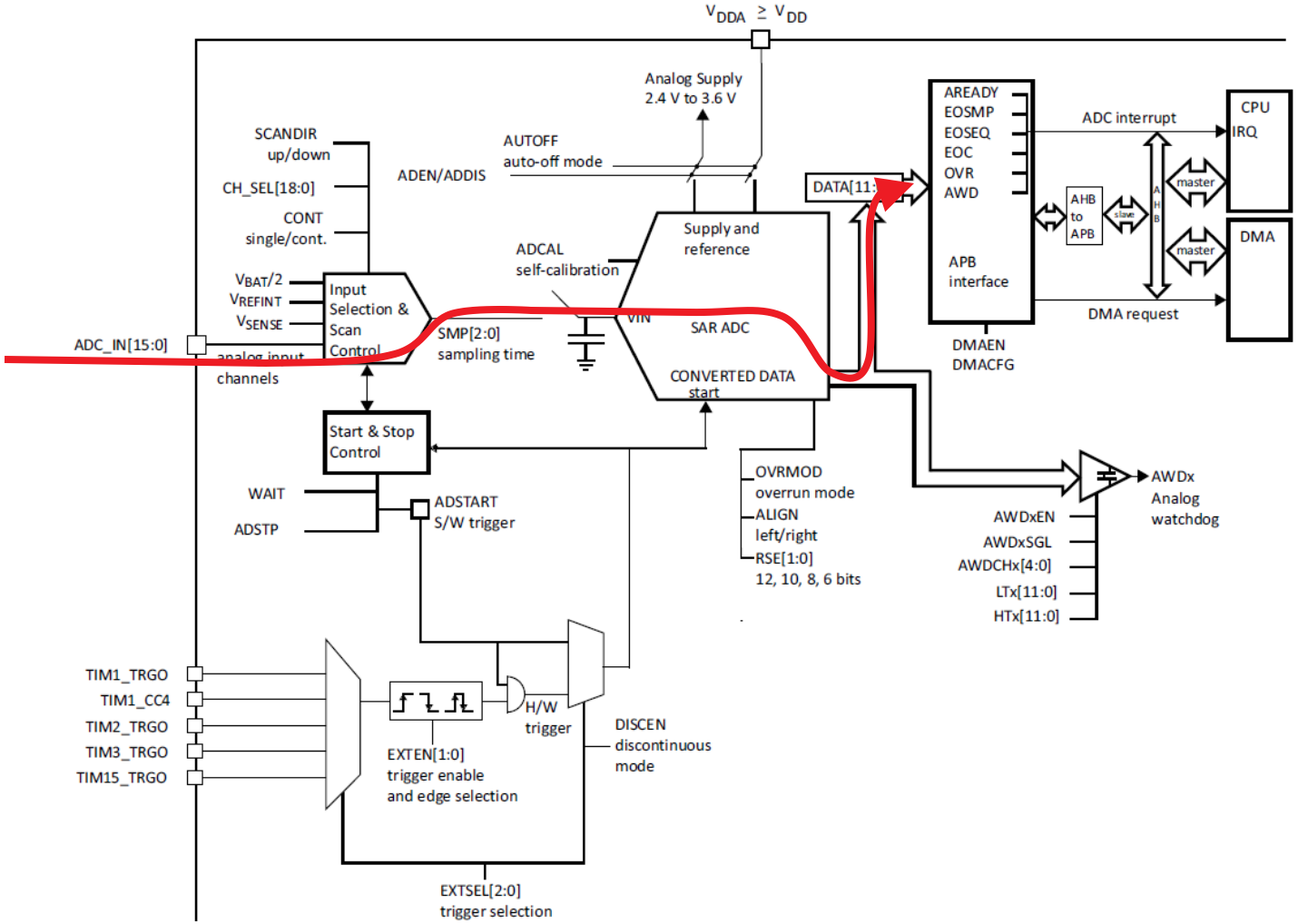
  - **Hoeveel Volt** stelt één LSB (Least Significant Bit) dan voor?

1 bit = 0,81 mV

hogeschool vives

AD-converter

- **Wat** is een AD-converter?



Bron: STM32F0 Reference Manual – RM0091 (st.com)

AD-converter

- **Instellingen** voor het gebruik van een AD-converter
    - Plaats **pin** A0 op **analoge mode**.
    - Voorzie een **klok** voor de AD-module.
    - Kies de **uitlijning**.
    - De AD-module **inschakelen**.
    - Kies een **kanaal** om te bemeten.
    - Eventueel **sample rate** kiezen.

AD-converter

- **Instellingen** voor het gebruik van een AD-converter

```
4  // AD-module initialiseren.
5  void InitAd(void)
6  {
7    // Poort A 0 als analoge ingang (potentiometer op het Nucleo Extension Shield).
8    GPIOA->MODER |= GPIO_MODER_MODER0;
9
10   // AD-module van klok voorzien.
11   RCC->APB2ENR |= RCC_APB2ENR_ADC1EN;
12
13   // Right alignment (optioneel).
14   ADC1->CFGR1 &= ~ADC_CFGR1_ALIGN;
15
16   // AD enable'n.
17   ADC1->CR |= ADC_CR_ADEN;
18   while ((ADC1->ISR & ADC_ISR_ADRDY) == 0);
19
20   // AD-kanaal 0 kiezen (verbonden met de connector voor analog input).
21   ADC1->CHSELR = ADC_CHSELR_CHSEL0;
22
23   // Tragere sample rate kiezen (voor hoogimpedante analoge bronnen).
24   ADC1->SMPR |= ADC_SMPR_SMP_0 | ADC_SMPR_SMP_1 | ADC_SMPR_SMP_2;
25 }
```

Zoek uit hoe alle bitwise operations werken.
Neem er zeker ook de **reference manual** bij!

AD-converter

- **Instellingen** voor het gebruik van een AD-converter

```
27 // De blokkerende versie van de AD-conversie (later met interrupts).
28 uint16_t GetAdValue(void)
29 {
30   uint32_t temp = 0;                          // Lokale variabele maken.
31
32   ADC1->CR |= ADC_CR_ADSTART;                 // Start de AD-omzetting.
33   while ((ADC1->ISR & ADC_ISR_EOSEQ) == 0);   // Wachten op het einde van de sequentie
34                                               // (sequentie bestaat hier uit één conversie).
35   ADC1->ISR |= ADC_ISR_EOSEQ;                 // EOSEQ-vlag wissen.
36   temp = ADC1->DR;                            // Dataregister uitlezen en opslaan.
37
38   return (uint16_t)temp;                      // De waarde van 'temp' casten en teruggeven.
39 }
```

Deze functie start de AD-omzetting en **keert terug met het resultaat**… Later meer over ‘functies’.

AD-converter

OPM: '>>' is 'shift right'...

- AD-omzetting **op de LED's**
  - De AD-omzetting is 12-bit.
  - De terugkeerwaarde van de `GetAdValue()` functie is 16-bit.
  - We hebben 8 LED's op ons NES...
  - **Hoe** kunnen we de AD-waarde toch mooi visualiseren **op de LED's**?

```
adValue = (uint8_t)(GetAdValue() >> 4);

ByteToLeds(adValue);
// of
//ByteToLevel(adValue);

WaitForMs(100);
```

Your turn now!

- Is er nog **wat resterende** tijd?
  - **Demo** – spanningsmeting met AD-converter.
  - Maak zelf **Theorieopdracht 1 – Algemene vragen**.
  - Ga **zelf aan de slag** om met in- en outputs te werken…